



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

**CÁTEDRA
DESARROLLO
DE SOFTWARE**



npmjs.com

*Gestor de paquetes, dependencias y proyectos en el ecosistema de JavaScript.
Base de datos con más de 2 millones de paquetes.*

Inicializar un proyecto

```
> npm init
```

Lanza un asistente en la terminal que nos preguntará sobre nuestro proyecto:

- nombre
- versión
- descripción
- entry point (main/index.js)
- comando de pruebas
- repositorio de git (URL del repositorio remoto, si ya existe)
- palabras clave (para buscadores)
- autor
- licencia

```
> npm init -y
```

La flag -y indica que se tomen todos los valores por default para el nuevo proyecto.

Esto generará un archivo **package.json** donde se encontrará toda la información que especificamos del proyecto, sus dependencias y scripts personalizados de ejecución:

```
{
  "name": "megasuper",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

Instalación de dependencias

```
> npm install <nombre del paquete>
```

Este comando instala el paquete y los demás paquetes de los que este depende en la carpeta **/node_modules** del proyecto (siempre debemos agregarla al `.gitignore`).

Se puede instalar una versión específica si se especifica de la siguiente forma:

```
> npm install <nombre del paquete>@<version>
```

Por ejemplo, ejecutar `npm install express` instalará el framework web Express. Además, al archivo `package.json` se agregará la siguiente entrada:

```
"dependencies": {  
  "express": "^5.1.0"  
}
```

Donde se registran las dependencias del proyecto y sus versiones necesarias. Existen distintas formas de especificar las versiones que se pueden ver en:

<https://docs.npmjs.com/cli/v11/configuring-npm/package-json#dependencies>

Además, se crea el archivo **package-lock.json** que especifica las versiones exactas de las dependencias (a diferencia del **package.json** que indica una referencia de rangos o versiones máximas y compatibles semánticamente).

También se pueden instalar **dependencias exclusivas para el desarrollo**, agregando el flag `--save-dev`, por ejemplo:

```
> npm install nodemon --save-dev
```

Agregará la siguiente entrada en el archivo `package.json`:

```
"devDependencies": {  
  "nodemon": "^3.1.9"  
}
```

En un proyecto que queremos correr, ya con sus dependencias especificadas en el `package.json` pero no instaladas, simplemente ejecutamos:

```
> npm install
```

Usamos la flag `--production` para ignorar las dependencias de desarrollo, que no serán necesarias en el entorno productivo.

Importación de Dependencias

Del mismo modo que importamos módulos dentro de nuestro mismo proyecto, en Node JS las dependencias se importan usando *'require'*.

```
const express = require('express')
```

ECMAScript Modules

Una forma, más novedosa, para importar módulos en Node. Sin ningún extra que evite esto, los paquetes más nuevos se importan de esta forma obligatoriamente.

El primer paso es, en nuestro *package.json*, hacer que el proyecto sea de tipo *"module"*.

```
"type": "module",
```

A partir de ahora, las dependencias se importarán de la siguiente manera:

```
import chalk from 'chalk';
```

Scripts

En la clave *"scripts"*, podemos agregar *alias* para los distintos comandos que vamos a estar usando para trabajar con nuestro proyecto. Estos son muy útiles para ejecutar distintas utilidades de desarrollo, tests o comandos específicos para diferentes ambientes.

Se especifican de la siguiente manera:

```
"scripts": {  
  "start": "node index.js",  
  "dev": "nodemon index.js"  
},
```

Para ejecutarlos, utilizaremos el siguiente comando:

```
> npm run <alias del script>
```

En el caso particular del alias *"start"* también se puede ejecutar directamente ejecutando:

```
> npm start
```